

Analizador Léxico y Sintáctico

Proyecto 2



Manual_Usuario

Universidad San Carlos de Guatemala

Facultad de ingeniería

Escuela de Ciencias y Sistemas

Introducción

Este manual técnico describe la arquitectura y el funcionamiento del compilador desarrollado en C++ y Qt para la lectura de scripts de tareas (.task). El documento detalla la implementación de un Autómata Finito Determinista (AFD) para el escaneo léxico y un Analizador Sintáctico Descendente Recursivo para validar la estructura del tablero. Se incluye la descripción de algoritmos, diagramas de clases y la lógica detrás de la generación de reportes dinámicos en HTML y la construcción del árbol de derivación en formato DOT. Esta guía constituye el sustento técnico para la validación y escalabilidad del sistema.

Instalación de Dependencias Para Ejecutar el Programa

Para ejecutar MedLexer, es necesario contar con el siguiente entorno de desarrollo:

Requisitos del Sistema:

Sistema Operativo: Windows 10/11, Linux o macOS.

Framework: Qt Framework 6.x o superior.

Compilador: MinGW 64-bit o MSVC 2019+.

Herramienta de Visualización: Graphviz (para visualizar archivos .dot).

Pasos para Compilar:

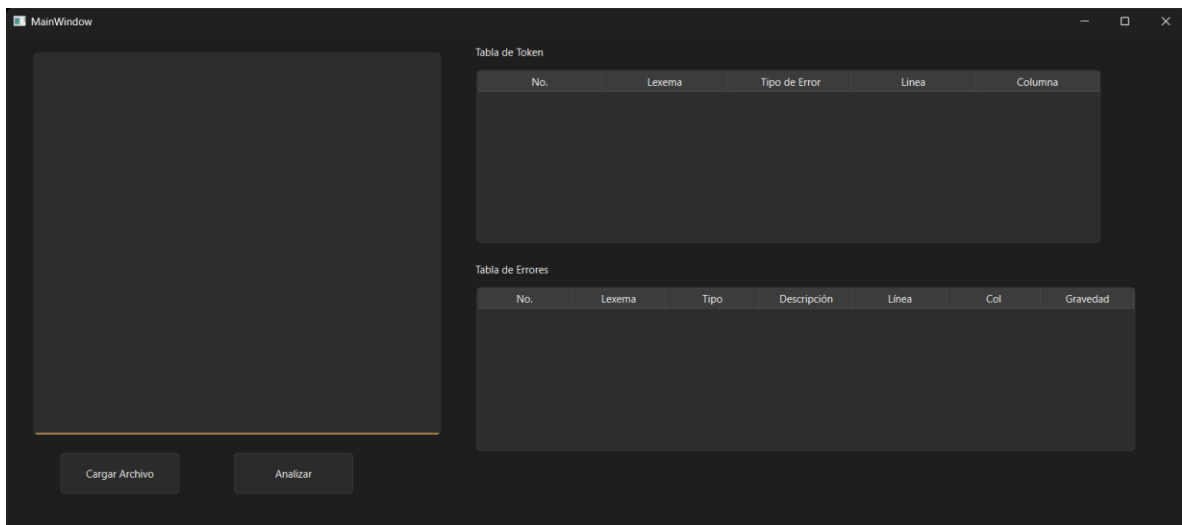
- Abrir el archivo MedLexer.pro en Qt Creator.
- Configurar el proyecto con el kit de compilación instalado (ej. Desktop Qt 6.5.0 MinGW).
- Presionar el botón "Run" (el icono del triángulo verde) o Ctrl + R.
- El ejecutable se generará en la carpeta de compilación (build folder).

Estructura de Archivos. fask

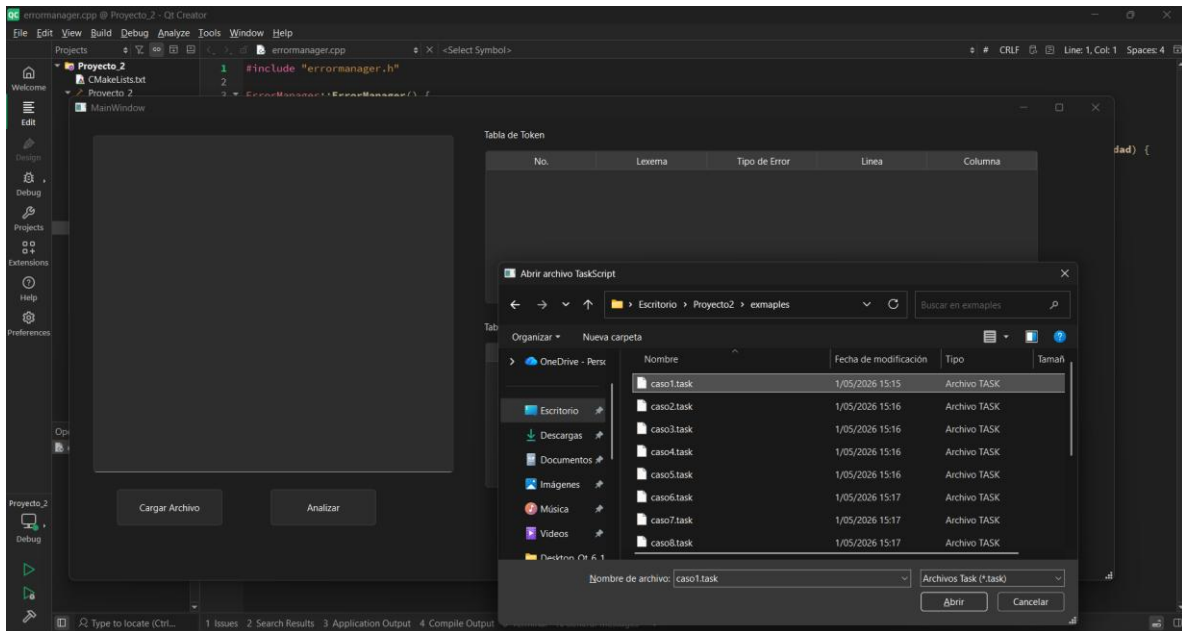
```
TABLERO "Proyecto LFP" {  
  COLUMNA "Por Hacer" {  
    tarea: "Diseñar AFD" [prioridad: ALTA, responsable: "Jorge",  
      fecha_limite: 2026-05-01],  
    tarea: "Implementar Lexer" [prioridad: ALTA, responsable: "Maria",  
      fecha_limite: 2026-05-08],  
    tarea: "Escribir casos de prueba" [prioridad: MEDIA,  
      responsable: "Carlos", fecha_limite: 2026-05-10],  
  };  
  
  COLUMNA "En Progreso" {  
    tarea: "Diseñar GUI" [prioridad: MEDIA, responsable: "Ana",  
      fecha_limite: 2026-05-05],  
  };  
  
  COLUMNA "Completado" {  
    tarea: "Investigar Qt" [prioridad: BAJA, responsable: "Pedro",  
      fecha_limite: 2026-04-20],  
    tarea: "Configurar GitHub" [prioridad: BAJA, responsable: "Jorge",  
      fecha_limite: 2026-04-18],  
  };  
};
```

Operaciones con el Programa

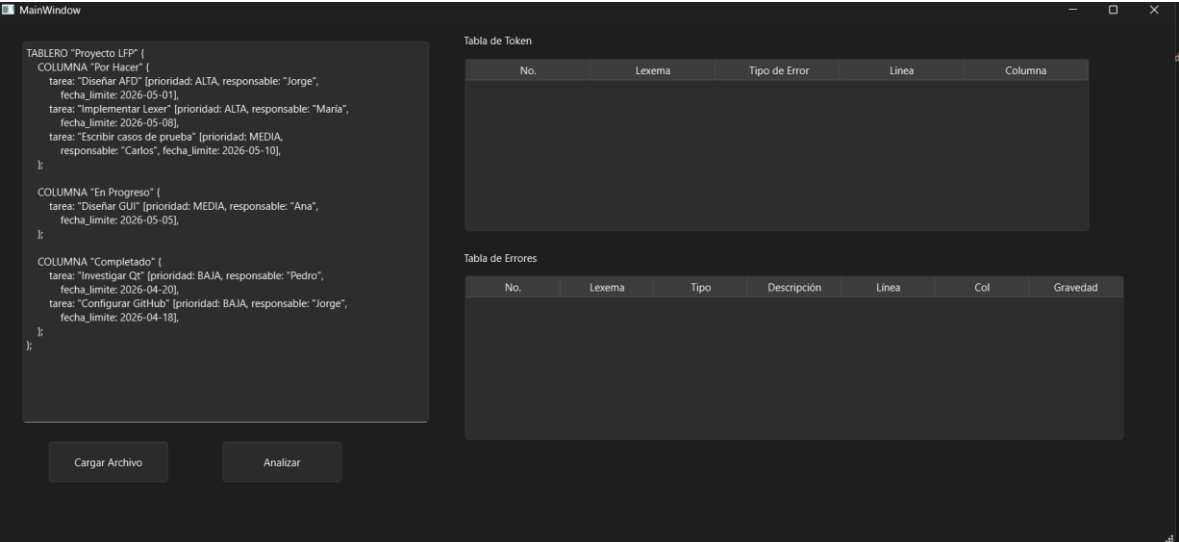
Interfaz de Programa



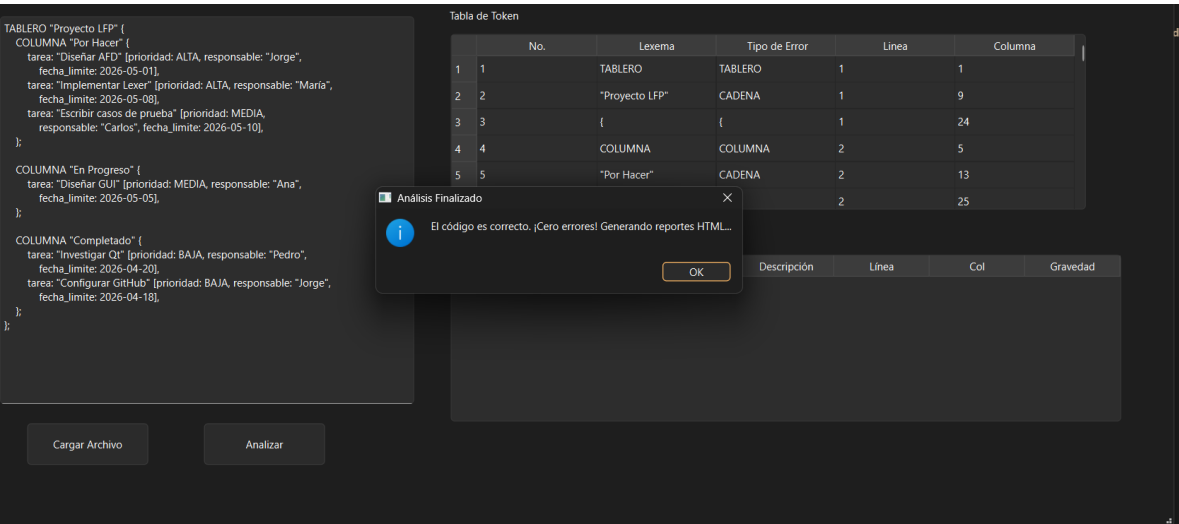
Cargando datos



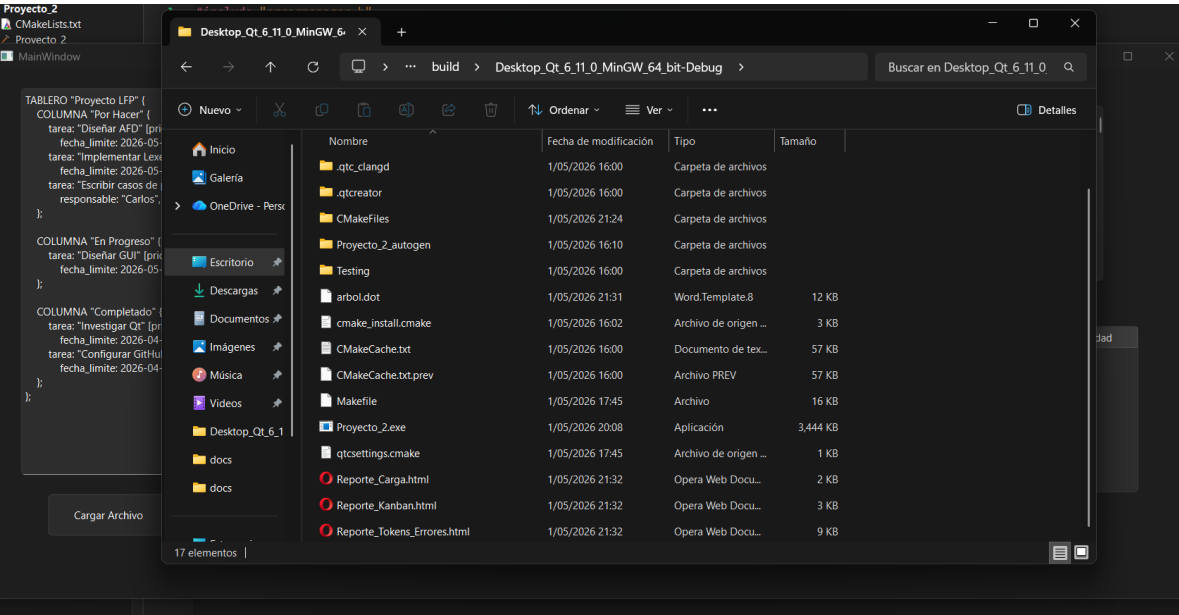
Datos Cargados exitosamente



Presionamos el botón analizar para que el programa inicie.



Observamos que nuestro programa funciona y genera los reportes y nuestro archivo .dot



Presionamos click en los enlaces web para ver los reportes que género.

El primer reporte que nos genero es el de responsabilidad.

Carga por Responsable					
Responsable	Tareas Asignadas	Alta	Media	Baja	Distribución
Ana	1	0	1	0	16%
Carlos	1	0	1	0	16%
Jorge	2	1	0	1	33%
Maria	1	1	0	0	16%
Pedro	1	0	0	1	16%

El segundo reporte es el de kamba, indicando las tareas por hacer catalogadas en la mas importante hasta la de menor importancia.

Proyecto LFP

Por Hacer (3)

Diseñar AFD

ALTA

Fecha límite: 2026-05-01

Jorge

Implementar Lexer

ALTA

Fecha límite: 2026-05-08

María

Escribir casos de prueba

MEDIA

Fecha límite: 2026-05-10

Carlos

En Progreso (1)

Diseñar GUI

MEDIA

Fecha límite: 2026-05-05

Ana

Completado (2)

Investigar Qt

BAJA

Fecha límite: 2026-04-20

Pedro

Configurar GitHub

BAJA

Fecha límite: 2026-04-18

Jorge

Vemos que el tercer reporte genera la tabla de errores como el archivo que analizamos no hay errores esta tabla estará vacía.

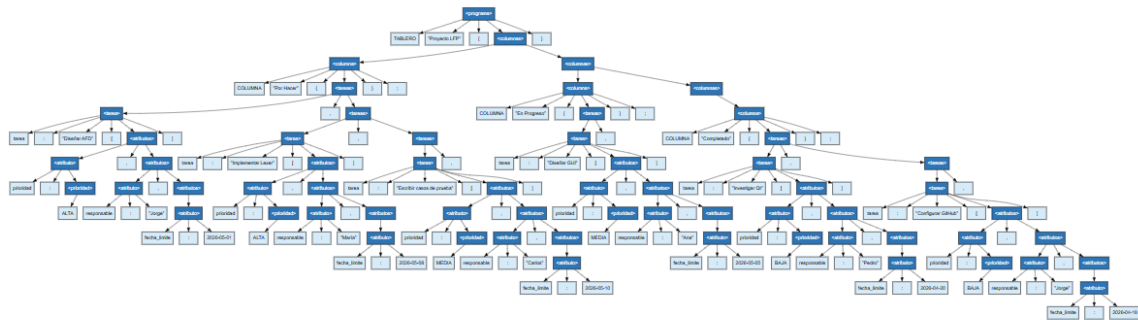
Tabla de Errores (0)

No.	Lexema	Tipo	Descripción	Línea	Columna	Gravedad
-----	--------	------	-------------	-------	---------	----------

Tabla de Tokens

No.	Lexema	Tipo	Línea	Columna
1	TABLERO	TABLERO	1	1
2	"Proyecto LFP"	CADENA	1	9
3	{	{	1	24
4	COLUMNA	COLUMNA	2	5
5	"Por Hacer"	CADENA	2	13
6	{	{	2	25
7	tarea	TAREA	3	9
8	:	:	3	14
9	"Diseñar AFD"	CADENA	3	16
10	[	[	3	31
11	prioridad	PRIORIDAD	3	32
12	:	:	3	41
13	ALTA	ALTA	3	43
14	,	,	3	47
15	responsable	RESPONSABLE	3	49

El programa también nos genera un archivo .dot este archivo nos sirve para ver el árbol de la estructura sintáctica del programa para poder ver lo copiamos el texto del archivo .dot que nos genero el programa y usamos una herramienta online de graphviz donde pegaremos el código que copiamos de nuestro archivo .dot



Ahora probaremos nuestro programa con un error lexico esto con el fin de probar que funciona adecuadamente.

En la interfaz cargamos nuestro archivo dañado.

TABLERO "Prueba Lexica" {

@

COLUMNA "Fase 1" {

tarea: "Ignorar simbolos" [prioridad: ALTA, responsable: "Jorge" -, fecha_limite: 2026-05-01]

};

\$

COLUMNA "Fase 2" {

tarea: "Prueba final" [prioridad: MEDIA, responsable: "Ana", fecha_limite: 2026-05-08]

};

}

Cargar Archivo

Analizar

Tabla de Token

No.	Lexema	Tipo de Error	Linea	Columna
-----	--------	---------------	-------	---------

Abrir archivo TaskScript

Escritorio > Proyecto2 > ejemplos

Buscar en ejemplos

Organizar Nueva carpeta

Nombre	Fecha de modificación	Tipo	Tamaño
caso1.task	1/05/2026 15:15	Archivo TASK	
caso2.task	1/05/2026 15:16	Archivo TASK	
caso3.task	1/05/2026 15:16	Archivo TASK	
caso4.task	1/05/2026 15:16	Archivo TASK	
caso5.task	1/05/2026 15:16	Archivo TASK	
caso6.task	1/05/2026 15:17	Archivo TASK	
caso7.task	1/05/2026 15:17	Archivo TASK	
caso8.task	1/05/2026 15:17	Archivo TASK	

Nombre de archivo: Archivos Task (*.task)

Abrir Cancelar

View to locate (Ctrl) 1. Issues 2. Search Results 3. Application Output 4. Compile Output

Al darle al botón analizar vemos que nos marca error de archivo, por ende no muestra el mensaje que se ha generado los reportes.

TABLERO "Prueba Lexica" {
@

COLUMNA "Fase 1" {
 tarea: "Ignorar simbolos" [prioridad: ALTA, responsable: "Jorge" ~, fecha_limite: 2026-05-01]
};

\$

COLUMNA "Fase 2" {
 tarea: "Prueba final" [prioridad: MEDIA, responsable: "Ana", fecha_limite: 2026-05-08]
};
}

Cargar Archivo

Analizar

Tabla de Token

	No.	Lexema	Tipo de Error	Línea	Columna
1	1	TABLERO	TABLERO	1	1
2	2	"Prueba Lexica"	CADENA	1	9
3	3	{	{	1	25
4	4	COLUMNA	COLUMNA	5	5
5	5	"Fase 1"	CADENA	5	13
		{	{	5	22

⚠️

Análisis Finalizado

Se encontraron errores. Revisa la tabla.

OK

	Tipo	Descripción	Línea	Col	Gravedad		
1		Carácter no ...	3	5	ERROR		
2	2	~	Léxico	Carácter no ...	6	74	ERROR
3	3	\$	Léxico	Carácter no ...	9	5	ERROR

Vemos que en nuestros archivos no se generan los reportes ni el archivo .dot, solo se genera el reporte de errores que nos muestra que fallos tuvo el archivo que le colocamos para analizar.

MakeLists.txt
Proyecto 2
MainWindow

TABLERO "Prueba Lexica" {
@

COLUMNA "Fase 1" {
 tarea: "Ignorar simbolos" [prioridad: ALTA, responsable: "Jorge" ~, fecha_limite: 2026-05-01]
};

\$

COLUMNA "Fase 2" {
 tarea: "Prueba final" [prioridad: MEDIA, responsable: "Ana", fecha_limite: 2026-05-08]
};
}

Cargar Archivo

Desktop_Qt_6.11.0_MinGW_64-bit-Debug

Buscar en Desktop_Qt_6.11.0

Nuevo

Inicio

Galería

OneDrive - Perso

Escritorio

Descargas

Documentos

Imágenes

Música

Videos

Desktop_Qt_6.1

docs

docs

exmaples

Nombre	Fecha de modificación	Tipo	Tamaño
.cmake	1/05/2026 16:00	Carpeta de archivos	
.qt	1/05/2026 17:45	Carpeta de archivos	
.qtc_clangd	1/05/2026 16:00	Carpeta de archivos	
.qtccreator	1/05/2026 16:00	Carpeta de archivos	
CMakeFiles	1/05/2026 21:24	Carpeta de archivos	
Proyecto_2_autogen	1/05/2026 16:10	Carpeta de archivos	
Testing	1/05/2026 16:00	Carpeta de archivos	
arbol.dot	1/05/2026 21:44	Word.Template8	5 KB
cmake_install.cmake	1/05/2026 16:02	Archivo de origen ...	3 KB
CMakeCache.txt	1/05/2026 16:00	Documento de tex...	57 KB
CMakeCache.txt.prev	1/05/2026 16:00	Archivo PREV	57 KB
Makefile	1/05/2026 17:45	Archivo	16 KB
Proyecto_2.exe	1/05/2026 20:08	Aplicación	3,444 KB
qtcsettings.cmake	1/05/2026 17:45	Archivo de origen ...	1 KB

14 elementos

Observamos que en el reporte nos marca los errores que contiene el archivo dañado.

Tabla de Errores (3)

No.	Lexema	Tipo	Descripción	Línea	Columna	Gravedad
1	@	Léxico	Carácter no reconocido por el lenguaje	3	5	ERROR
2	~	Léxico	Carácter no reconocido por el lenguaje	6	74	ERROR
3	\$	Léxico	Carácter no reconocido por el lenguaje	9	5	ERROR

Tabla de Tokens

No.	Lexema	Tipo	Línea	Columna
1	TABLERO	TABLERO	1	1
2	"Prueba Lexica"	CADENA	1	9
3	{	{	1	25
4	COLUMNA	COLUMNA	5	5
5	"Fase 1"	CADENA	5	13
6	{	{	5	22
7	tarea	TAREA	6	9
8	:	:	6	14
9	"Ignorar simbolos"	CADENA	6	16
10	[	[	6	35
11	prioridad	PRIORIDAD	6	36
12	.	.	6	46

Con esto concluimos que el programa funciona adecuadamente, si ingresamos un archivo sin errores el programa generar los reportes sin ningún problema y el archivo .dot, pero al momento de ingresar un archivo dañado el programa nos indicara que el archivo tiene errores y no generar mas que el reporte de errores y que errores contiene el archivo que le ingresamos.

Conclusiones

Eficacia del Modelo Híbrido: La combinación de un Autómata Finito Determinista (AFD) para el análisis léxico y un Analizador Sintáctico Descendente Recursivo demostró ser una metodología altamente eficiente para procesar el lenguaje TaskScript. Esta estructura permite una clasificación precisa de lexemas y una validación jerárquica de la gramática con una complejidad temporal óptima de $O(n)$.

Gestión Integral de Errores: La implementación de una estrategia diferenciada ante fallos garantiza la estabilidad del sistema. Mientras que la recuperación léxica permite al usuario corregir múltiples símbolos inválidos en una sola pasada, la detención fatal ante errores sintácticos asegura la integridad de los reportes generados, cumpliendo estrictamente con las normativas de validación técnica.

Valor de la Visualización de Datos: La integración de herramientas de visualización como Graphviz (DOT) y reportes dinámicos en HTML5 transforma scripts de texto plano en información estructurada y comprensible. Esto facilita la auditoría del proceso de compilación mediante el árbol de derivación y optimiza la gestión de proyectos al presentar tableros Kanban y estadísticas de carga de forma visual.